day_01RAG项目概述

项目背景

近年来，随着ChatGPT的广泛应用，基于大规模语言模型（LLM）的技术已成为人工智能领域的研究和应用热点。尤其是大模型在各类自然语言处理任务中的成功应用，推动了教育行业的智能化转型。然而，当前市面上大多数大语言模型存在一个普遍的问题：这些模型主要依赖于过往的训练数据，无法动态获取最新的知识以及各企业特有的私有知识。这种局限性常常导致生成答案时出现“幻觉”问题，即模型提供的答案与实际情况不符或不准确。

为了解决以上问题我们提出两种方式解决：

第一种：基于企业私有知识的垂直领域微调。通过专业领域的特定知识融合到大模型中（训练），使得模型能够更好的服务于垂直行业的专业需求。

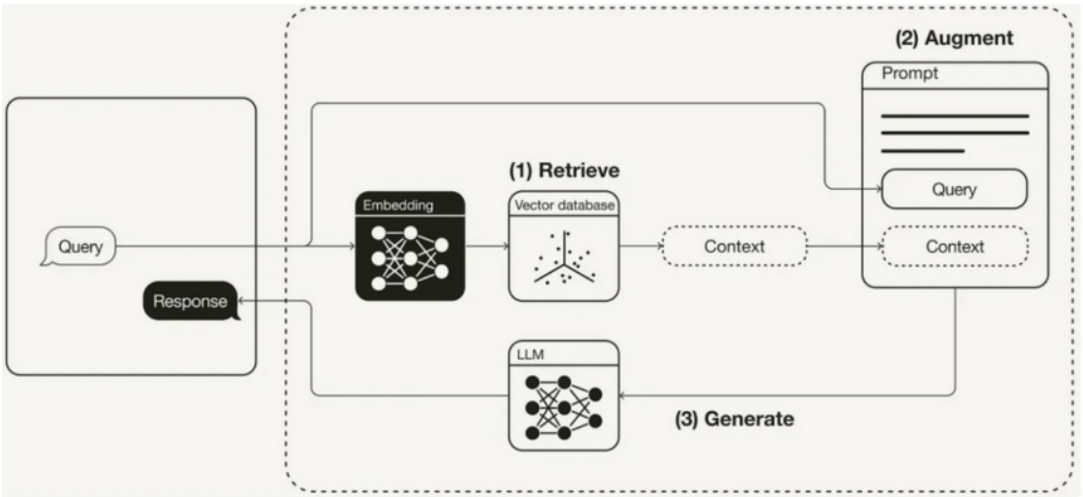
第二种：基于企业私有知识的RAG问答系统，通过构建检索的问答框架，结合企业私有知识库，更精准动态更新知识服务。

我们往往使用第二种，因为微调成本极高，可能需要多张类似于H100的顶尖显卡交火训练，且训练周期极长且生效很慢，且依赖于大量人工标注数据，而且工程师的数量要求也比RAG要多得多。

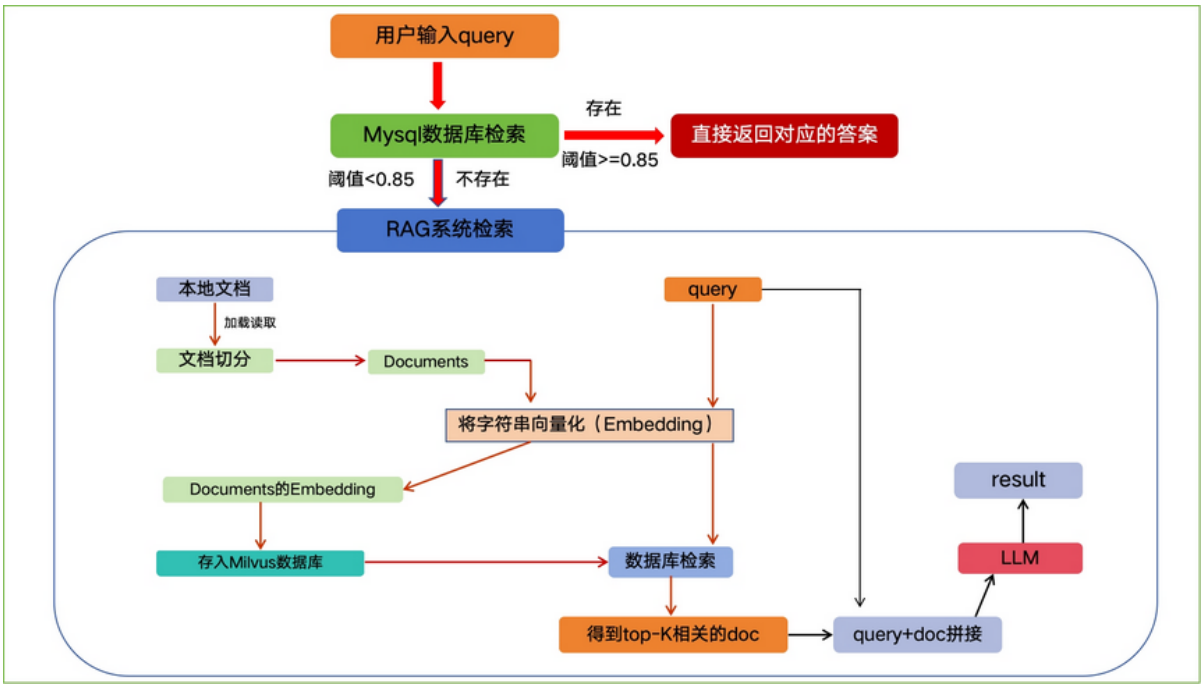
通过构建基于检索的问答框架，结合企业私有知识库，实现更为精准且动态更新的知识问答服务，从而减少幻觉问题的发生。

项目原理

RAG是Retrieve Augment Generate的缩写，意思是检索增强生成



项目流程



首先用户先输入问题也就是query，然后从MySQL数据库中检索，该数据库中存储高频问答对，并通过redis进行缓存，如命中缓存则直接返回，如果没有则去MySQL中查找。通过BM25（TFIDF的改进）进行关键字匹配，如果大于所设定的阈值则返回对应的答案，如果低于阈值则从RAG系统检索，构建RAG系统也要分两步，首先要有对应的文档进行切分，转化为向量后存入向量数据库，然后用户的问题query也转换为向量进入数据库查找匹配，一般是使用复杂查找RFFranker重排序查找最终将得到的top-k相关的文档与原始query拼接后交给大模型并得到最终的结果。

Milvus 向量数据库

一款开源得向量数据库，存储、索引和管理由深度神经网络和其它机器学习模型生成的大规模嵌入向量。能够处理万亿级别的向量索引，主要处理非结构化数据。Milvus通过计算向量之间的相似距离分析两个向量之间的关联性，如果两个嵌入向量非常相似则往往代表原始数据也很相似。

Milvus的强大之处：处理大规模的向量时候性能极高，开发者社区活跃，提供多语言支持和工具链，扩展性很强

非结构数据

非结构数据一般指的是文本，音频，视频，图像等数据，但是这是狭义上的，广义上的非结构数据指的是不能通过Key Value形式存储在关系型数据库如MySQL中的数据，哪怕是音频视频图像等，如果存储他们的地址引用或者编码到MySQL数据库中，这也能算是结构化数据，虽然它们本身依然是非结构化数据，但是我们能以某种方式结构化存储。

向量相似度查询

将向量与数据库进行比较，查找最相似的向量，一般使用kmeans算法加速搜索过程。

Collection和Field

一个Milvus中最多支持64个向量数据库，Collection和Field可以对应关系型数据库中的表和字段，1个collection最多支持4个向量Field。

Milvus	关系数据库	描述
Collection	表	集合相当于关系数据库中的表，用于组织数据
Field	字段	字段Schema相当于表中的列
is_primary	主键	在Field Schema中标记为主键对应该列的主键
dtype	数据类型	字段的数据类型，如INT, VARCHAR等
max_length	最大长度	对应VARCHAR类型字段的最大字符数
dim	-	向量字段的维度没有直接对应，但可以视为特殊数据处理

Field Schema

Field schema是字段的逻辑定义。我们在定义集合架构和管理集合之前需要定义的第一件事就是定义Field schema 。

属性	描述	备注
`name`	要创建的集合中的字段名称	String，必填
`dtype`	字段的数据类型	必填
`description`	字段描述	String，选填
`is_primary`	是否设置该字段为主键字段	Boolean (`true` or `false`) 主键字段必填
`auto_id` （主键字段必填）	切换以启用或禁用自动 ID（主键）分配	`True` 或 `False`
`max_length` （VARCHAR 字段必需）	允许插入的字符串的最大长度。	`[1, 65,535]`
`dim`	向量的维数	`∈[1, 32768]`
`is_partition_key`	该字段是否是分区键字段	布尔值（`true` 或 `false`）

Collection Schema

collection schema 是 collection 的逻辑定义。我们需要在定义 collection schema 之前定义 field schema。

属性	描述	备注
`field`	集合中要创建的字段	必填
`description`	集合描述	String,选填
`partition_key_field`	设计用作分区键的字段的名称。	String， 选填
`enable_dynamic_field`	是否启用动态模式	Boolean (`true` or `false`)

Milvus支持的索引类型

索引是数据的组织单位，在搜索和查询插入的实体前必须声明索引类型和相似度度量，不声明的话将使用暴力搜索。

索引类型

FLAT：适合在小型也就是百万级的数据集上寻求完全准确和精确搜索结果，其实本质就是一种暴力搜索，保证精确度，在大数据量时候效果很差，除非要求绝对精准的情况下如医疗。

IVF_FLAT：一种基于倒排的索引，通过kmeans先将高维空间的向量划分为多个簇，每个簇都是相似的向量，然后为每个簇创建倒排索引，每个向量都属于某一个簇。查询时候首先与簇中心进行对比匹配，然后在簇内再逐个进行对比向量。使用参数nprobe来控制搜索的簇数量，增大簇数量能够更精准但是时间更长。

IVF_SQ8：基于IVF_FLAT但是引入了量化，本质思想与IVF_FLAT是一致的，但是通过标量量化，将每个维度四个字节(FLOAT32)转化为一个字节(INT8)表示，损失了一定的精度，毕竟都被取整了，但是速度大大提高了，且CPU友好。

IVF_PQ：一种高效的向量索引方式，结合了倒排文件索引和乘积量化。在倒排索引时候也是会分为多个簇并查询这些向量与聚类中心的距离，选择最接近的几个簇进行详细搜索，减少计算量。但是在每个簇内，向量又被分为了多个子向量，通过独立量化过程进行编码，这样子保持了较高处理速度和准确性。

对于百万内的数据直接使用FLAT最佳，其余直接使用IVF_PQ

HNSW：HNSW是基于图的索引，最适合对搜索效率有高要求的场景。

相似度度量

一般判断相似度有三种方法

第一种：欧几里得距离，就是计算坐标相减后的平方和再开方，两点之间的距离公式，取值范围0到+∞

第二种：内积，两个向量做元素乘法（相乘再相加），因为向量有正有负，向量元素乘法如果为负说明向量方向相反，为0说明正交无相关性，为正说明两个向量相似，且值越大越相似，也就是说两个向量长度越长且方向越一致就会越相关，取值范围为常数范围

第三种：余弦相似度，公式为

$$\text{cosine_similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

其中||A||与||B||为模长，整个公式取值范围为-1到1，此方式是计算两个向量夹角，所以与向量长度无关，夹角越小越相似，如果向量正交则相似度为0，如果为负数则表示负相关，最不相似。

设置向量数据库

代码块

```
def operate_db():
# 如果uri为数据库名称路径，代表本地操作数据库
# client = MilvusClient(uri="milvus_demo.db")
# 如果uri为链接地址，代表Milvus属于单机服务，需要开启Milvus后台服务操作
client = MilvusClient(uri="http://localhost:19530")
# 创建名称为milvus_demo的数据库
databases = client.list_databases()
if "milvus_demo" not in databases:
    client.create_database(db_name="milvus_demo")
else:
    client.using_database(db_name="milvus_demo")
return client
```

Collections操作

在 Milvus 中，我们需要一个 Collections 来存储向量及其相关元数据。你可以把它想象成传统 SQL 数据库中的表格。创建 Collections 时，可以定义 Schema 和索引参数来配置向量规格，如维度、索引类型和远距离度量。此外，还有一些复杂的概念来优化索引以提高向量搜索性能。

client.create_schema()其实一个创建表的类，我们这里创建了一个对象schema接收

schema.add_field是添加字段的意思这里是显式使用schema来创建，可指定多个字段，指定其数据类型，维度和名字

client.create_collection()才是真正的建表，这时候要内部传入前面定义的字段也就是schema对象

index_params也是client.prepare_index_params()的一个对象用来生成索引

这里使用add_index()内部传入要增加索引的字段名字，如果是向量还要定义metric_type相似度搜索方式(内积，欧式距离，余弦相似度)index_type索引类型(FLAT,IVF_FLAT.IVF_SQ8,IVF_PQ)如果不传默认是FLAT暴力检索，但是这里也是依然是定义创立索引的方式，真正的创建还是发生在client.create_index()中

如果是VARCHAT类型的字段标量，那么创建索引时候index_type一般是不传东西的

代码块

```
# 2 collection集合的操作
def operate_table():
    # 定义schema
    ## 注意：在定义集合 Schema 时，enable_dynamic_field=True 使得您可以插入未定义的字段。一般动态字段以 JSON 格式存储，通常命名为 $meta。在插入数据时，所有未定义的字段及其值将被保存为键值对。
    ## 在定义集合 Schema 时，auto_id=True 可以对主键自动增长id。
    schema = client.create_schema(auto_id=False, enable_dynamic_field=True)
    # # schema添加字段id, vector
    schema.add_field(field_name='id', datatype=DataType.INT64, is_primary=True)
    schema.add_field(field_name='vector', datatype=DataType.FLOAT_VECTOR, dim=5)
    schema.add_field(field_name='scalar1', datatype=DataType.VARCHAR, max_length=256, description='标量字段')

    # # 创建集合
    client.create_collection(collection_name='demo_v1', schema=schema)
    # # 设置索引
    index_params = client.prepare_index_params()
    # # 在向量字段vector上面添加一个索引;
    # index_type='',  # 留空以使用自动索引
    # 对于向量字段，常见的默认索引类型包括IVF_FLAT或HNSW等，具体取决于数据的特性和查询需求。
    # 对于标量字段，常见的默认索引可能是INVERTED等。
    index_params.add_index(field_name='vector', metric_type="COSINE", index_type='', index_name="vector_index")
    client.create_index(collection_name='demo_v1', index_params=index_params)
    #
    # # 查看索引信息
    res = client.list_indexes(collection_name='demo_v1')
    print(f'索引信息--》{res}')

    res = client.describe_index(collection_name='demo_v1', index_name='vector_index')
    print(f'指定索引详细信息-->{res}')

    # 查看索引状态
    # client.load_collection(collection_name='demo_v1')
    # print(client.get_load_state(collection_name='demo_v1'))
    # 如果不需要索引，可以删除相关索引
    # client.release_collection(collection_name='demo_v1')
    # client.drop_index(collection_name='demo_v1', index_name='vector_index')

```

```
    # # 检索标量字段
    index_params1 = client.prepare_index_params()
    index_params1.add_index(field_name='scalar1', index_type='',
index_name='default_index')
    client.create_index(collection_name='demo_v1', index_params=index_params1)
    # #
    # # # 查看索引信息
    res = client.list_indexes(collection_name='demo_v1')
    print(f'索引信息--》{res}')
    #
    res = client.describe_index(collection_name='demo_v1',
index_name='vector_index')
    print(f'指定索引详细信息-->{res}')

```

Entity实体数据操作

在 Milvus 中，**实体指的是Collections中共享相同Schema** 的数据记录，行中每个字段的数据构成一个实体。因此，同一 Collections 中的实体具有相同的属性（如字段名称、数据类型和其他约束）。

数据的增删改

注意要小心使用auto_id=True和enable_dynamic_field=True,因为milvus不像关系型数据MySQL之类的生成的主键是1，2，3，4的形式，它生成的是一个有规律的大数类似462141304663713546，这个不可能记得住，所以会导致删除困难，必须先通过查询别的字段得到id甚至进行向量的相似度搜索来删除，特别麻烦enable_dynamic_field=True则在某些版本上有问题，动态字段无法使用upsert进行更新，但是在新版本中已经升级

client.create_collection(collection_name='demo_v2', dimension=5, metric_type='IP')这种collection表创建方法与之前的schema不同，它是会以一个默认模板也就是{id:0,vector:0.11111}这样的形式创建collection，并在此指定维度查询方式，比如这里设定是内积的形式查询相似度，默认使用的是auto_id=False和enable_dynamic_field=True，使用这种方式创建的除id和vector外的字段都是动态字段，无法指定index_type，只有显式使用schema的才行

代码块

```
def operate_entity():
    # # todo:1. 创建集合collection
    # 这种方式: collection 只包括两个字段. id 作为主键， vector 作为向量字段，以及自动设置 auto_id、enable_dynamic_field 为 True
    # auto_id 启用此设置可确保主键自动递增。在数据插入期间无需手动提供主键。
    # enable_dynamic_field 启用后，要插入的数据中除 id 和 vector 之外的所有字段都将被视为动态字段。
    # # 这些附加字段作为键值对保存在名为 $meta 的特殊字段中。此功能允许在数据插入期间包含额外的字段。
```

```
    client.create_collection(collection_name='demo_v2', dimension=5, metric_type='IP')
    #
    # # todo:2. 插入数据（也叫实体）
    data = [
        {"id": 0, "vector": [0.3580376395471989, -0.6023495712049978, 0.18414012509913835, -0.26286205330961354,
                             0.9029438446296592], "color": "pink_8682"},
        {"id": 1, "vector": [0.19886812562848388, 0.06023560599112088, 0.6976963061752597, 0.2614474506242501,
                             0.838729485096104], "color": "red_7025"},
        {"id": 2, "vector": [0.43742130801983836, -0.5597502546264526, 0.6457887650909682, 0.7894058910881185,
                             0.20785793220625592], "color": "orange_6781"},
        {"id": 3, "vector": [0.3172005263489739, 0.9719044792798428, -0.36981146090600725, -0.4860894583077995,
                             0.95791889146345], "color": "pink_9298"},
        {"id": 4, "vector": [0.4452349528804562, -0.8757026943054742, 0.8220779437047674, 0.46406290649483184,
                             0.30337481143159106], "color": "red_4794"},
        {"id": 5, "vector": [0.985825131989184, -0.8144651566660419, 0.6299267002202009, 0.1206906911183383,
                             -0.1446277761879955], "color": "yellow_4222"},
        {"id": 6, "vector": [0.8371977790571115, -0.015764369584852833, -0.31062937026679327, -0.562666951622192,
                             -0.8984947637863987], "color": "red_9392"},
        {"id": 7, "vector": [-0.33445148015177995, -0.2567135004164067, 0.8987539745369246, 0.9402995886420709,
                             0.5378064918413052], "color": "grey_8510"},
        {"id": 8, "vector": [0.39524717779832685, 0.4000257286739164, -0.5890507376891594, -0.8650502298996872,
                             -0.6140360785406336], "color": "white_9381"},
        {"id": 9, "vector": [0.5718280481994695, 0.24070317428066512, -0.3737913482606834, -0.06726932177492717,
                             -0.6980531615588608], "color": "purple_4976"}
    ]
    res = client.insert(collection_name='demo_v2', data=data)
    print(res)

    ## todo:2.1 将数据插入到特定分区，可以在插入请求中指定分区名称，如下所示:
    data = [
        {"id": 10, "vector": [-0.5570353903748935, -0.8997887893201304, -0.7123782431855732, -0.6298990746450119,
                              0.6699215060604258], "color": "red_1202"},
        {"id": 11, "vector": [0.6319019033373907, 0.6821488267878275, 0.8552303045704168, 0.36929791364943054,
                              -0.14152860714878068], "color": "blue_4150"},
```

```
        {"id": 12, "vector": [0.9483947484855766, -0.32294203351925344,
0.9759290319978025, 0.8262982148666174,
                              -0.8351194181285713], "color": "orange_4590"},
        {"id": 13, "vector": [-0.5449109892498731, 0.043511240563786524,
-0.2510524948479080 4, -0.012030655265886425,
                              -0.0010987671273892108], "color": "pink_9619"},
        {"id": 14, "vector": [0.6603339372951424, -0.10866551787442225,
-0.9435597754324891, 0.8230244263466688,
                              -0.7986720938400362], "color": "orange_4863"},
        {"id": 15, "vector": [-0.8825129181091456, -0.9204557711667729,
-0.935350065513425, 0.5484069690287079,
                              0.24448151140671204], "color": "orange_7984"},
        {"id": 16, "vector": [0.6285586391568163, 0.5389064528263487,
-0.3163366239905099, 0.22036279378888013,
                              0.15077052220816167], "color": "blue_9010"},
        {"id": 17, "vector": [-0.20151825016059233, -0.905239387635804,
0.6749305353372479, -0.7324272081377843,
                              -0.33007998971889263], "color": "blue_4521"},
        {"id": 18, "vector": [0.2432286610792349, 0.01785636564206139,
-0.651356982731391, -0.35848148851027895,
                              -0.7387383128324057], "color": "orange_2529"},
        {"id": 19, "vector": [0.055512329053363674, 0.7100266349039421,
0.4956956543575197, 0.24541352586717702,
                              0.4209030729923515], "color": "red_9437"}
    ]

    # ##  todo:3. 创建分区
    client.create_partition(collection_name='demo_v2',
partition_name='partitionA')
    # # # # todo: 3.1 分区中插入数据
    res = client.insert(collection_name='demo_v2', data=data,
partition_name='partitionA')
    print(res)
    ## todo:4. 更新插入数据
    # 在 Milvus 中，upsert 操作执行数据级操作，根据集合中是否已存在主键来插入或更新实
体。具体来说：
    # 如果集合中已存在该实体的主键，则现有实体将被覆盖。
    # 如果集合中不存在主键，则将插入一个新实体。
    data = [
        {"id": 0, "vector": [-0.619954382375778, 0.4479436794798608,
-0.17493894838751745, -0.4248030059917294,
                             -0.8648452746018911], "color": "black_9898"},
        {"id": 1, "vector": [0.4762662251462588, -0.6942502138717026,
-0.4490002642657902, -0.628696575798281,
                             0.9660395877041965], "color": "red_7319"},
        {"id": 2, "vector": [-0.8864122635045097, 0.9260170474445351,
0.801326976181461, 0.6383943392381306,
```

```
                                0.7563037341572827],"color": "white_6465"},
        {"id": 3, "vector": [0.14594326235891586, -0.3775407299900644,
-0.3765479013078812, 0.20612075380355122,
                                0.4902678929632145], "color": "orange_7580"},
        {"id": 4, "vector": [0.4548498669607359, -0.887610217681605,
0.5655081329910452, 0.19220509387904117,
                                0.016513983433433577], "color": "red_3314"},
        {"id": 5, "vector": [0.11755001847051827, -0.7295149788999611,
0.2608115847524266, -0.1719167007897875,
                                0.7417611743754855], "color": "black_9955"},
        {"id": 6, "vector": [0.9363032158314308, 0.030699901477745373,
0.8365910312319647, 0.7823840208444011,
                                0.2625222076909237], "color": "yellow_2461"},
        {"id": 7, "vector": [0.0754823906014721, -0.6390658668265143,
0.5610517334334937, -0.8986261118798251,
                                0.9372056764266794], "color": "white_5015"},
        {"id": 8, "vector": [-0.3038434006935904, 0.1279149203380523,
0.503958664270957, -0.2622661156746988,
                                0.7407627307791929], "color": "purple_6414"},
        {"id": 9, "vector": [-0.7125086947677588, -0.8050968321012257,
-0.32608864121785786, 0.3255654958645424,
                                0.26227968923834233], "color": "brown_7231"}
    ]

    res = client.upsert(collection_name='demo_v2', data=data)
    print(res)
    # 注意如果分区中不存在更新数据的id，就不会受影响，但是会影响集合里已经存在的相同id的
实体
    res = client.upsert(collection_name='demo_v2', data=data,
partition_name="partitionA")
    # todo:5. 删除实体（数据）
    # 按照过滤器删除；如果不指定分区，默认情况下会在整个集合中进行删除
    res = client.delete(collection_name='demo_v2', filter='id in [12, 5, 6]')
    print(res)
    # 按照id进行删除；指定分区删除数据
    res = client.delete(collection_name='demo_v2', ids=[1, 2, 3, 4],
partition_name='partitionA')
    print(res)

```

数据的查询

简单查询

除了指定为FLAT外其它查找方式都是ANN（近似最近邻，不是全连接神经网络）

```python
# entity实体数据的操作：查询
def query_operation():
    # # todo: 1. 单向量搜索
    res = client.search(collection_name='demo_v2',
                        data=[[0.19886812562848388, 0.06023560599112088, 0.6976963061752597, 0.2614474506242501, 0.838729485096104]],
                        limit=2,
                        search_params={"metric_type": "IP"},
                        output_fields=["id", 'vector']) # search_params是在查询时执行距离计算方式，如果定义索引的时候，已经制定了方式可以不写
    print(res)
    # todo: 2. 批量向量搜索
    res = client.search(collection_name='demo_v2',
                        data=[[0.19886812562848388, 0.06023560599112088, 0.6976963061752597, 0.2614474506242501, 0.838729485096104],
                              [0.3172005263489739, 0.9719044792798428, -0.36981146090600725, -0.4860894583077995, 0.95791889146345]],
                        limit=2,
                        search_params={"metric_type": "IP"},
                        output_fields=["id", 'vector']) # search_params是在查询时执行距离计算方式，如果定义索引的时候，已经制定了方式可以不写
    print(res)
    # todo: 3. 分区搜索
    # 要进行分区搜索，只需在搜索请求的 partition_names 中包含目标分区的名称即可。这指定search操作仅考虑指定分区内的向量。
    res = client.search(
        collection_name="demo_v2",
        data=[[0.02174828545444263, 0.058611125483182924, 0.6168633415965343, -0.7944160935612321, 0.5554828317581426]],
        limit=5,
        search_params={"metric_type": "IP", "params": {}},
        partition_names=["partitionA"]  # 这里指定搜索的分区
    )
    print(res)
    # todo: 4.使用输出字段进行搜索
    # 使用输出字段进行搜索允许您指定搜索结果中应包含匹配向量的哪些属性或字段。
    res = client.search(
        collection_name="demo_v2",
        data=[[0.3580376395471989, -0.6023495712049978, 0.18414012509913835, -0.26286205330961354, 0.9029438446296592]],
        limit=5,
        search_params={"metric_type": "IP", "params": {}},
        output_fields=['vector', "color"]  # 返回定义的字段
    )
    print(res)
    # todo: 5.过滤搜索
    # 过滤器搜索：筛选搜索将标量筛选器应用于矢量搜索，允许我们根据特定条件优化搜索结果。
```

```
    # 例如，要根据字符串模式优化搜索结果，可以使用 like 运算符。此运算符通过考虑前缀、中缀和后缀来启用字符串匹配：
    # 筛选颜色以红色为前缀的结果：
    res = client.search(
        collection_name="demo_v2",
        data=[[0.3580376395471989, -0.6023495712049978, 0.18414012509913835, -0.26286205330961354, 0.9029438446296592]],
        limit=5,
        search_params={"metric_type": "IP", "params": {}},
        output_fields=["color"],
        filter='color like "red%"'
    )
    print(res)
    # todo: 6.范围搜索
    # 范围搜索允许查找距查询向量指定距离范围内的向量。
    # 范围搜索:radius：定义搜索空间的外边界。只有距查询向量在此距离内的向量才被视为潜在匹配。
    # range_filter：虽然radius设置搜索的外部限制，但可以选择使用range_filter来定义内部边界，创建一个距离范围，在该范围内向量必须落下才被视为匹配。
    search_params = {
        "metric_type": "IP",
        "params": {
            "radius": 0.8,  # 搜索圆的半径
            "range_filter": 1  # 范围过滤器，用于过滤出不在搜索圆内的向量。
        }
    }

    res = client.search(
        collection_name="demo_v2",
        data=[[0.3580376395471989, -0.6023495712049978, 0.18414012509913835, -0.26286205330961354, 0.9029438446296592]],
        limit=3,  # 返回的搜索结果最大数量
        search_params=search_params,
        output_fields=["color"],
    )
    #
    #
    print(res)
    result = json.dumps(res, indent=4)
    print(result)

```

复杂查询

在RAG中通常使用两种向量，稀疏向量和稠密向量，稠密向量信息量大，一般使用bert之类模型生成，擅长语义理解，稀疏向量由bm25和tfidf生成，用来关键词匹配，milvus可在coloection中存储两种向

量并执行多路ANN搜索

要对两组ANN搜索结果进行合并重新排序，要有适当的重新排序策略，支持两种重排策略。

第一种：加权排名策略，通过WeightedRanker可以为某些向量场分配更高的权重，需要在WeightedRanker中输入权重值，输入次数和搜索次数要一致，输入数值为0到1，和可以不为0，越接近1表示越重要，比如对稀疏和稠密的向量生成的相似度得分进行加权，我们设定稠密向量的权重为0.8稀疏向量的为0.3这样子即可

$$finanscore(d) = w_1 \cdot norm(score_{\text{dense}}) + w_2 \cdot norm(score_{\text{sparse}})$$

w1,w2是人工设置的权重，norm表示归一化，因为稠密和稀疏分数范围不同所以要归一化才能相加

优点：直观可控，可简单调整权重完成，适合你知道哪种向量更重要的情况

缺点：依赖调参，需要大量实验才能确定，容易偏向某一路向量，丧失互补性

代码块

```
from pymilvus import WeightedRanker
rerank = WeightedRankder(0.8,0.3)
```

第二种：最常用的还是RRFRankder倒数排序融合，在没有特定的侧重点情况下使用，可以有效平衡每个向量场的重要性。核心思想是根据每个结果在其检索列表中的排名位置来计算分数

$$score(d) = \sum_{i=1}^{N} \frac{1}{k + rank_i(d) + 1}$$

d表示文档，N表示不同检索路径的数量，ranki(d)表示d在第i个检索器的排名位置，从0开始计数，k是一个平滑参数来让输出更平滑，一般设置为60

通过计算稠密向量和稀疏向量的得分排名然后通过公式求和得到，排名越靠前分母越小分数越大

文档	稠密排名	稀疏排名	RRF 得分（k=60）
DocA	1	2	1/61 + 1/62 ≈ 0.0325
DocB	2	1	1/62 + 1/61 ≈ 0.0325
DocC	1	10	1/61 + 1/70 ≈ 0.0306

由图可知DocA和DocB虽然得分的来源排名不同但是分数一样所以都排第一名，因为它只关注排名，不关注具体分数是如何来的

优点：无需调参，公平融合，比起加权算法，它不偏向任意一种向量，鲁棒性更强，适合多模态跨语言的集成场景

代码块

```
def complex_query():
    # # 定义schema

    schema = client.create_schema(enable_dynamic_field=False)
    schema.add_field(field_name='film_id', datatype=DataType.INT64, is_primary=True)
    schema.add_field(field_name='filmVector', datatype=DataType.FLOAT_VECTOR, dim=5) # 向量字段
    schema.add_field(field_name="posterVector", datatype=DataType.FLOAT_VECTOR, dim=5) # 向量字段
    # #
    # 定义索引
    index_params = client.prepare_index_params()
    index_params.add_index(field_name='filmVector', index_type="IVF_FLAT",
                           metric_type="L2", params={"nlist": 128})
    index_params.add_index(field_name='posterVector', index_type="",
                           metric_type="COSINE")

    # 创建集合
    client.create_collection(collection_name='demo_v3', schema=schema, index_params=index_params)

    # 向量库中插入实体
    entities = []
    for  _ in range(1000):
        # 构造实体
        film_id = random.randint(1, 10000)
        film_vector = [random.random() for _ in range(5)]
        poster_vector = [random.random() for _ in range(5)]
        entity = {"film_id": film_id, "filmVector": film_vector, "posterVector": poster_vector}
        entities.append(entity)

    client.insert(collection_name='demo_v3', data=entities)

    # 多向量查询（注意和批量向量查询不同）
    # 多向量搜索使用 hybrid_search() API 在一次调用中执行多个 ANN 搜索请求。每个 AnnSearchRequest 代表特定矢量场上的单个搜索请求。
    # 示例创建两个 AnnSearchRequest 实例以对两个向量字段执行单独的相似性搜索。
    # 创建多搜索请求 filmVector
    query_filmVector = [[0.8896863042430693, 0.370613100114602, 0.23779315077113428, 0.38227915951132996, 0.5997064603128835]]
```

```
    dense_search_params = {"data": query_filmVector,
                           "anns_field": "filmVector",# 该参数值必须与集合模式中使用的值相同。
                           "param": {"metric_type": "L2", "nprobe": 10},# nprobe代表访问簇的数量
                           "limit": 2}
    request_1 = AnnSearchRequest(**dense_search_params)

    # 创建多搜索请求 posterVector
    query_posterVector = [[0.02550758562349764, 0.006085637357292062, 0.5325251250159071, 0.7676432650114147, 0.5521074424751443]]
    sparse_search_params = {"data": query_posterVector,
                            "anns_field": "posterVector",
                            # 该参数值必须与集合模式中使用的值相同。
                            "param": {"metric_type": "COSINE"},
                            "limit": 2
                            }
    request_2 = AnnSearchRequest(**sparse_search_params)

    reqs = [request_1, request_2]
    ranker = RRFRanker(100)

    res = client.hybrid_search(
        collection_name="demo_v3",
        reqs=reqs,
        ranker=ranker,
        limit=2
    )
    for hits in res:
        print("TopK results:")
        for hit in hits:
            print(hit)

```

Collection的删除

代码块

```
# Drop collection
client.drop_collection(collection_name="demo_collection")
```